## JS Object และ JSON

Nodejs ใช้ JSON เป็นรูปแบบในการแลกเปลี่ยนข้อมูล แต่ระหว่างการประมวลผลภายในโปรแกรมจะใช้ JavaScript Object

1. **JSON (JavaScript Object Notation)** เป็นรูปแบบข้อมูลที่ใช้ในการแลกเปลี่ยนข้อมูลระหว่าง client-server หรือระหว่างแอปพลิเคชันต่าง ๆ โดย JSON เป็นรูปแบบข้อความ (text-based format) ที่อ่านง่ายสำหรับมนุษย์และเครื่องจักร (เครื่องคอมพิวเตอร์) และเป็นที่นิยมมากในการสื่อสารข้อมูลใน **API** หรือ **HTTP requests** โดยเฉพาะในแอปพลิเคชันที่พัฒนาโดยใช้ **Node.js**

   ตัวอย่างการใช้ JSON ในการแลกเปลี่ยนข้อมูล:

   - เมื่อรับ-ส่งข้อมูลระหว่าง Client กับ Server (เช่น ผ่าน REST API)
   - การจัดเก็บข้อมูลในฐานข้อมูล NoSQL อย่าง MongoDB (เก็บข้อมูลในรูปแบบ JSON-like document)

   ตัวอย่าง JSON:

```
{
    "name": "John Doe",
    "age": 30,
    "email": "john@example.com"
}
```

2. **JavaScript Object**: ภายในโปรแกรม Node.js (หรือใน JavaScript ทั่วไป) จะใช้ JavaScript Object เป็นโครงสร้างข้อมูลหลักในการประมวลผล ซึ่ง JavaScript Object มีโครงสร้างคล้าย JSON มาก แต่ยืดหยุ่นกว่า เช่น คุณสามารถใช้ฟังก์ชันหรือค่าอื่น ๆ ที่ไม่รองรับใน JSON ได้

   เมื่อทำงานใน Node.js หลังจากที่ได้รับข้อมูล JSON จาก Client หรือจาก API ต่าง ๆ โปรแกรมจะต้องแปลงข้อมูล JSON ให้เป็น JavaScript Object โดยใช้ฟังก์ชัน JSON.parse() เพื่อให้สามารถประมวลผลข้อมูลภายในโปรแกรมอย่างสะดวก เช่น อ่านค่า แก้ไขข้อมูล หรือดำเนินการทาง logic ต่าง ๆ

ตัวอย่าง JavaScript Object:

```
const user = {
  name: "John Doe",
  age: 30,
  email: "john@example.com",
  greet: function() {
    console.log(`Hello, my name is ${this.name}`);
  }
};

user.greet();  // Output: Hello, my name is John Doe
```

**สรุป**

- **JSON**: ใช้สำหรับการแลกเปลี่ยนข้อมูลระหว่างระบบ (เช่น ระหว่าง client-server) เพราะเป็นรูปแบบที่เหมาะสำหรับการส่งผ่านเครือข่ายและการจัดเก็บ
- **JavaScript Object**: ใช้ภายในโปรแกรม Node.js ในการประมวลผลข้อมูล เมื่อรับ JSON เข้ามา โปรแกรมมักจะแปลง JSON เป็น JavaScript Object เพื่อความสะดวกในการใช้งาน

ฟังก์ชัน JSON.parse() ใช้แปลงข้อมูล JSON ให้เป็น JavaScript Object แต่ถ้าต้องการการส่งข้อมูลใน JavaScript Object ไปยังภายนอก Node.js เราต้องใช้คำสั่งอย่างไร

กรณีส่งข้อมูลในรูปแบบ JavaScript Object ออกไปยังภายนอก Node.js (เช่น ส่งข้อมูลผ่าน API หรือเขียนลงไฟล์) คุณจำเป็นต้องแปลง JavaScript Object ให้เป็น JSON โดยใช้ฟังก์ชัน JSON.stringify() ซึ่งจะเปลี่ยน JavaScript Object ให้เป็น JSON string (ข้อความ JSON) ที่สามารถส่งออกได้ในรูปแบบที่เหมาะสม

กรณีต้องการส่งข้อมูลจาก Node.js ไปยัง client (เช่นเว็บเบราว์เซอร์หรือแอปพลิเคชันอื่น ๆ) คุณสามารถใช้ JSON.stringify() เพื่อแปลง JavaScript Object ให้เป็น JSON string ก่อนส่งออกในรูปแบบ HTTP response ดังตัวอย่างต่อไปนี้

1. **การส่งข้อมูลผ่าน API (HTTP Response)**: การส่งข้อส่งข้อมูลจาก Node.js ไปยัง client หรือ แอปพลิเคชันอื่น เพื่อใช้แลกเปลี่ยนข้อมูลกันไปมา การใช้ข้อมูลในรูปแบบ JSON เป็นมาตรฐานที่เหมาะสมในการแลกเปลี่ยนข้อมูล โดยสามารถใช้ JSON.stringify() เพื่อแปลง JavaScript Object ให้เป็น JSON string ก่อนส่งออกในรูปแบบ HTTP response ดังตัวอย่างต่อไปนี้

```
const express = require('express');
const app = express();

app.get('/user', (req, res) => {
    // JavaScript Object ที่ต้องการส่งออก
    const user = {
        name: "John Doe",
        age: 30,
        email: "john@example.com"
    };

    // แปลง JavaScript Object เป็น JSON string แล้วส่งกลับ client
    res.send(JSON.stringify(user));
});

app.listen(3000, () => {
    console.log('Server is running on port 3000');
});
```

หรือใน Express คุณสามารถใช้ res.json() แทน JSON.stringify() ซึ่งจะทำงานเหมือนกันโดยอัตโนมัติ

```
app.get('/user', (req, res) => {
    const user = {
        name: "John Doe",
        age: 30,
        email: "john@example.com"
```

```
    };
    res.json(user);  // ส่ง JSON string โดยไม่ต้องเรียก JSON.stringify() เอง
});
```

2. **การเขียนข้อมูลลงไฟล์**: หากต้องการบันทึกข้อมูล JavaScript Object ลงในไฟล์ การเลือกบันทึกข้อมูลลงไปในไฟล์ในรูปแบบ JSON เป็นทางเลือกที่เหมาะสม โดยที่ก่อนการบันทึกต้องทำการแปลงข้อมูลที่ต้องการด้วยการใช้ JSON.stringify() ร่วมกับ fs.writeFile() หรือ fs.writeFileSync() จากโมดูล fs ของ Node.js ดังตัวอย่างต่อไปนี้

```
const fs = require('fs');

const user = {
    name: "John Doe",
    age: 30,
    email: "john@example.com"
};

// แปลง JavaScript Object เป็น JSON string และเขียนลงไฟล์
fs.writeFile('user.json', JSON.stringify(user, null, 2), (err) => {
    if (err) throw err;
    console.log('Data written to file');
});
```

ในตัวอย่างนี้จะเห็นว่า

- JSON.stringify(user, null, 2) แปลง user เป็น JSON string โดยจัดรูปแบบให้มีการย่อหน้าสวยงาม (ค่าที่ 2 คือจำนวนช่องว่างในการ indent)
- ไฟล์ user.json จะถูกสร้างขึ้นพร้อมข้อมูลที่ถูกแปลงจาก JavaScript Object เป็น JSON

ใช้ **JSON.stringify()** เพื่อแปลง **JavaScript Object** ให้เป็น **JSON string** เมื่อคุณต้องการส่งข้อมูลออกไปภายนอก เช่น ส่งข้อมูลผ่านไปยัง API หรือบันทึกลงไฟล์ สำหรับการส่งข้อมูลผ่าน **API** คุณสามารถใช้ res.json() ใน Express.js ซึ่งจะช่วยแปลง JavaScript Object เป็น JSON string โดยอัตโนมัติ